

safa.ai — Production Readiness Report

An AI film-generation product: a Next.js web app + a background render worker, tested end-to-end against its live database, storage, and AI providers.

Prepared 12 July 2026 · Scope: **safa-web** (web app) & **ai-film-pro** (render worker) · Tester ran install, typecheck, production build, and a full live generation of a film from script to finished MP4.

DONE

works & verified

STRONG

solid, production-quality

NEEDS FIX

fix before launch

BLOCKER

must fix first

1. Verdict

The product works. I ran a real film generation locally against your live Neon database, Cloudflare R2 bucket, OpenAI, and fal.ai: a script became a 4-shot breakdown, generated a character look, painted keyframes, filmed 4 clips with Seedance, stitched them with ffmpeg, watermarked the free-plan film, uploaded it to R2, and served it on a public share page. Auth, credit-spend, ownership checks, admin protection, and the Stripe webhook signature check all behaved correctly.

It is not launch-ready yet because of a small number of concrete blockers — the most serious being **live secrets committed into the repo** (including in the `.env.example` template) and a **stale 445 MB duplicate folder** shipping alongside the real app. None are deep architecture problems; they are a focused day or two of hardening. Details and a checklist follow.

2. What the project contains

safa-web — the web app STRONG

Next.js 14 (App Router) · TypeScript strict · Prisma + Postgres (Neon) · NextAuth · Stripe · Resend

- Pages:** landing, pricing, login/signup, forgot/reset password, account & billing, workspace (`/app`), admin console (`/admin`), public share page (`/p/[token]`), custom 404 + error pages.
- 22 API routes:** register, verify (+resend), forgot/reset, me, projects CRUD, breakdown/options/select/clips/share/chat, AI script "expand", Stripe checkout/portal/webhook, admin overview/credits, NextAuth.
- Auth:** email+password (bcrypt) with enforced email verification, optional Google sign-in, JWT sessions, middleware protecting `/app` and `/admin`.
- Billing:** Stripe subscription checkout, customer portal, webhook that sets plan + credits.
- Cost control:** per-user render credits, in-memory rate limiting, capped chat-driven regeneration.

Also in the folder: `safa-web-old/` (a stale earlier copy — see blocker B4) and two `.zip` archives of both projects.

ai-film-pro — the render worker STRONG

Node + tsx · Prisma (shared DB) · fal.ai · OpenAI · AWS S3 SDK → R2 · ffmpeg

- Worker** (`src/worker.ts`): polls the shared DB for queued jobs and runs each pipeline phase, writing progress back.
- Pipeline steps:** 1-breakdown, 2-options, 3-sheet, 4-images (keyframes), 5-videos (Seedance), 6-assemble (ffmpeg stitch).
- Job types:** breakdown, regen_shot, options, shots, regen_clip, assemble — each resumable and cached.
- Storage:** uploads options, keyframes, clips, and final film to R2; refunds a credit if a render fails.
- Extras:** free-plan watermark overlay, standalone local preview server (`src/server.ts`), CLI stage scripts.

3. Tests I ran & results

CHECK	RESULT	NOTES
Dependency install (both projects)	PASS	Clean install on Node 20; ffmpeg 8.1.2 present.
TypeScript typecheck	1 error → fixed	Invalid Stripe <code>apiVersion</code> broke the build; I corrected it (see A1).
Production build (<code>next build</code>)	PASS	All 24 routes compile after the fix; worker typechecks clean.
Server boot + all pages	PASS	Every public page returns 200; unknown share token returns 404.

Auth: register → verify → login → session	PASS	Email verification enforced; unverified login blocked as designed.
Access control (auth, ownership, admin)	PASS	/app + /admin redirect when signed out; APIs 401; other users' projects 404; admin APIs 403 for non-admins.
Stripe webhook signature verification	PASS	Unsigned webhook correctly rejected with 400.
Rate limiting (register/resend)	PASS	Register throttles after 5 attempts/min. (Gap: not on all routes — see A3.)
Full generation, script → finished film	PASS	Real run: breakdown, 1 character option, 4 keyframes, 4 Seedance clips, ffmpeg stitch, watermark, R2 upload, public playback. Credit spent correctly (3→2).
Malformed request bodies	weak	Invalid JSON returns 500 instead of 400 (see A2).
npm audit	4 vulns	3 moderate + 1 critical, all in next /transitive deps (see A4).

4. Issues to work on

Launch blockers BLOCKER

- B1 — Live secrets are committed in the repo, including the .env.example template.**

Both .env files and the .env.example templates contain real, working credentials: the Neon Postgres URL & password, an OpenAI key, a fal.ai key, Cloudflare R2 access key + secret, a Resend key, and a Google OAuth client secret. .env.example is meant to be committed, so these are effectively public. **Anyone with the repo can run up your OpenAI/fal bills, read/wipe your database, and read/overwrite your media bucket.**

Fix: Treat every one of these keys as compromised — **rotate all of them now** (Neon, OpenAI, fal, R2, Resend, Google). Replace .env.example with placeholders only (DATABASE_URL="postgresql://..."). Confirm .env is git-ignored (it is) and set real values only in the host's environment (Vercel / Railway / Render).
- B2 — Stripe is not configured.** STRIPE_SECRET_KEY , price IDs, and the webhook secret are still sk_test_... / price_... placeholders. Paid upgrades cannot work, and the "upgrade" flow will fail for real users. **Fix:** create the products/prices in Stripe, fill the real keys, register the webhook at /api/stripe/webhook , and enable the customer portal (steps in PRODUCTION.md).
- B3 — In-memory rate limiting won't hold in production.** The limiter is a per-process Map (lib/ratelimit.ts). On Vercel (serverless, many instances) it resets constantly and is trivially bypassed, so the auth/abuse protection you have is effectively off in production. **Fix:** back it with Upstash Redis (or a WAF) before launch — this is also the single most common VAPT finding.
- B4 — A stale 445 MB duplicate app ships in the folder.** safawebold/ is an older copy with its own .env full of the same live secrets. It should never reach a repo or a server. **Fix:** delete safawebold/ and the two .zip archives before you push/deploy.

Should fix before launch NEEDS FIX

- A1 — Stripe API version (already fixed by me).** lib/stripe.ts pinned apiVersion: "2024-06-20" , which the installed Stripe types reject — this failed the production build. I changed it to "2025-02-24.acacia" and the build now passes. Verify against your Stripe account's version.
- A2 — Malformed JSON returns HTTP 500.** Routes call await req.json() unguarded, so bad input throws and surfaces as a 500 (register, projects, and others). Wrap parsing in try/catch and return 400. Best paired with proper input validation (Zod) on every route body.
- A3 — Regeneration endpoints have no credit cost and no rate limit.** /api/projects/[id]/breakdown (regenerate/regenShot) and /api/projects/[id]/options (regenerate) queue real fal.ai/LLM work but are free and uncapped — a logged-in user can loop them and run up your provider bill. Chat-driven regen is metered, but these direct routes are not. Add a credit charge or a per-project cap.
- A4 — npm audit : 1 critical + 3 moderate** (all in next 14.2.15 and transitive postcss / uuid). Upgrade to a patched Next.js 14.2.x (or a vetted newer line) and re-run; add Dependabot/CI audit.

A5 — No content moderation on generated media. You are liable for what users generate; there is no moderation or abuse-report path. Add a moderation pass and a report route before opening signups broadly.

Known gaps for scale & hardening **PLANNED**

Your own `PRODUCTION.md` already tracks these honestly. Highlights that matter most for "handle real traffic": a real job queue with row-locked claims so you can run multiple workers (today it's single-worker DB polling with no lock — two workers would grab the same job); retries with backoff on failed renders; idempotent Stripe webhooks (store processed event IDs); signed/expiring URLs for private media instead of a fully public R2 bucket; per-user spend caps; monitoring (Sentry + uptime); Postgres backups/PITR; and a third-party pen-test. These are the difference between "works" and "survives launch day."

5. Master checklist

ITEM	STATUS
Both projects install & typecheck	DONE
Production build passes (after Stripe fix)	DONE
End-to-end film generation (script → MP4)	DONE
Auth, email verification, sessions	STRONG
Authorization / ownership / admin gating	STRONG
Stripe webhook signature verification	STRONG
Credit spend + refund-on-failure	STRONG
Security headers (HSTS, nosniff, frame, etc.)	STRONG
Public share flow + custom 404/error pages	DONE
Rotate all leaked secrets; clean <code>.env.example</code>	BLOCKER
Configure real Stripe keys / prices / webhook	BLOCKER
Durable (Redis) rate limiting	BLOCKER
Delete <code>safa-web-old/</code> + <code>.zip</code> archives	BLOCKER
Return 400 on malformed JSON; add Zod validation	NEEDS FIX
Meter/cap breakdown & options regeneration	NEEDS FIX
Resolve <code>npm audit</code> (upgrade Next.js)	NEEDS FIX
Content moderation + abuse reporting	NEEDS FIX
Version mismatch: CHANGELOG says 1.0.0, package.json 0.9.5	NEEDS FIX
Real job queue + multi-worker row locking	PLANNED
Retries/backoff, idempotent webhooks, backups	PLANNED
Signed media URLs, spend caps, monitoring, pen-test	PLANNED

Report generated after a live end-to-end test run. One code change was made during testing: `safa-web/lib/stripe.ts` Stripe `apiVersion` corrected so the production build passes. All test data (throwaway account, project, generated media) was deleted from your database and local disk afterward. "PLANNED" items are drawn from your own `PRODUCTION.md`, which already documents them.